

# Sistemas Operativos

Introdução e Conceitos

1

1

## Objetivos

- Evolução histórica
- Hardware (HW) e Sistema Operativo (SO)
- Funções do SO (interface e gestão de recursos)
- Mono vs Multiprogramação
- Tipos e Estruturas de SO

2

2

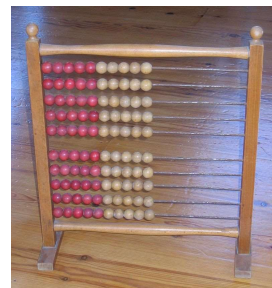
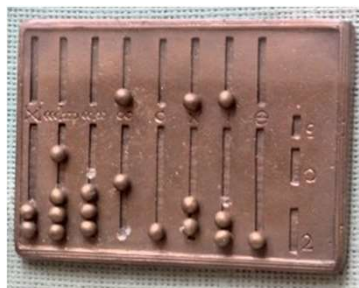
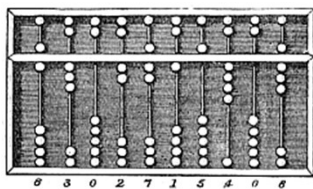
# Evolução histórica

3

3

## Evolução histórica

- Primeira calculadora: Ábaco
- Origem Mesopotâmia 4500 anos a.C



4

4

## Evolução histórica

- Computador Mecânico
- i.e componentes mecânicos (alavancas e engrenagens)

1620's



Wilhelm Schickard



1950's



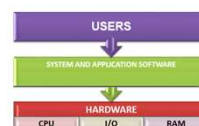
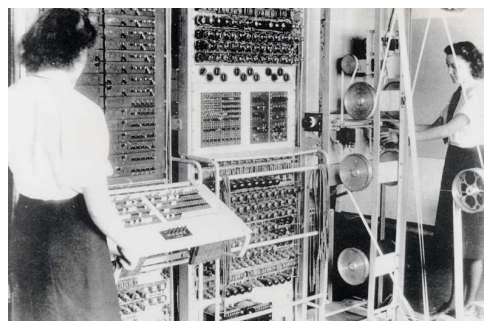
Hamann Manus R

5

5

## Evolução histórica

- 1945
  - 1ª Geração
  - Sistemas a Válvulas
  - Sem SO
  - Linguagem máquina (0 1)
- E.g. Colossus Mark 1 e 2

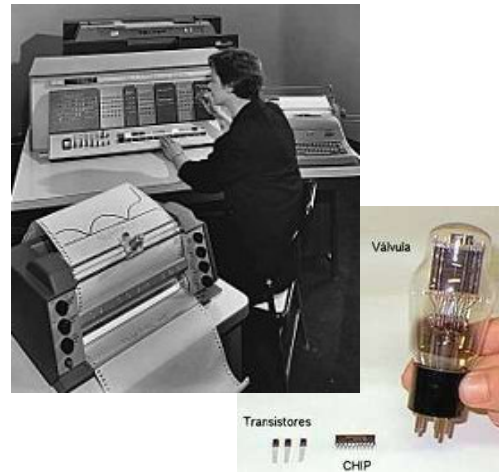


6

6

## Evolução histórica

- 1955
  - **2ª Geração**
  - **Transístores**
    - Pequenos
    - Não aquecem tanto
  - SO execução por lotes
  - Linguagem assembly (MOV) e fortran
- E.g. IBM 1620 (cartões perfurados)



7

## Evolução histórica

### Tratamento em lotes

- Trabalhos (jobs) são submetidos em ordem sequencial para a execução
- não existe interação durante a execução
- cada job pode ter várias tarefas

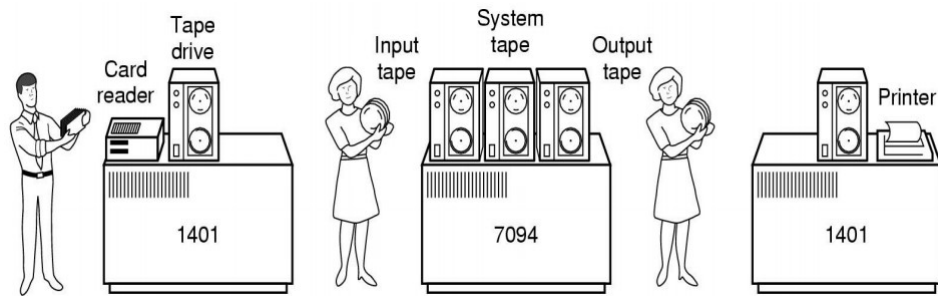


8

## Evolução histórica

### Cartões perfurados

programas escritos por um programador que era entregue ao operador da máquina para que o mesmo fosse processado.

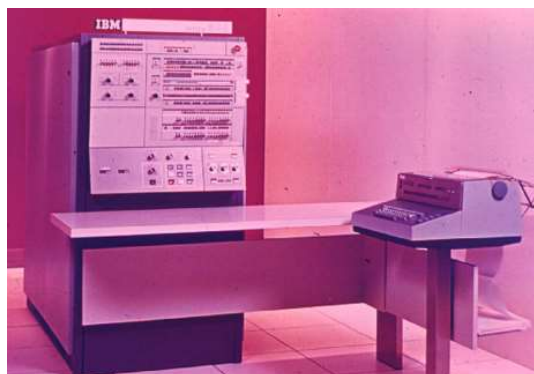


9

9

## Evolução histórica

- 1965
  - **3ª Geração**
  - **Circuitos integrados**
  - Multiprogramação
  - Tempo partilhado
- E.g. IBM-360 series



10

10

## Evolução histórica

- 1980
  - **4ª Geração**
  - **Microcomputadores**
    - DOS
    - Windows
    - Unix
  - Redes e **Internet**



11

11

## Evolução histórica

- 1990
  - **5ª Geração**
  - Sistemas distribuídos
  - Cliente/Servidor
  - Sistemas tempo real
  - IOT
  - Sistemas embebidos



12

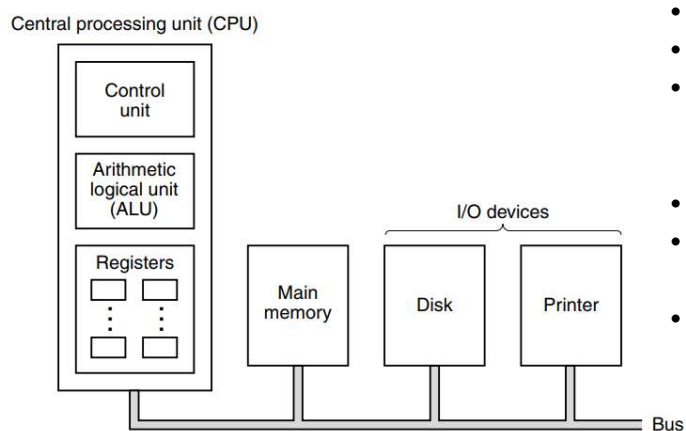
12

# HW e SO

13

13

## Organização de um sistema computacional



- 1 ou mais CPUs
- Memória principal (o que é ?)
- E/S (ports)
  - estender funcionalidades
- Muitos processos em execução...
- Poucos recursos disponíveis...
- ...boa gestão de recursos por parte do SO (como?)

14

14

# Alternativa ao SO?

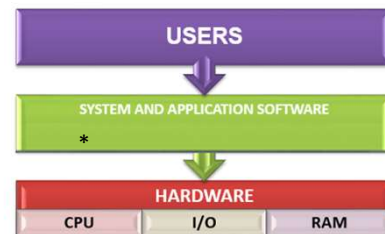
## Ausência de SO

Cada aplicação tem de ter também um tradutor para comunicar com o HW. \*

Maior esforço e dificuldade de programação

Trabalho repetido

Sem políticas globais de segurança, tolerância a falhas, otimização



15

15

# Alternativa ao SO?

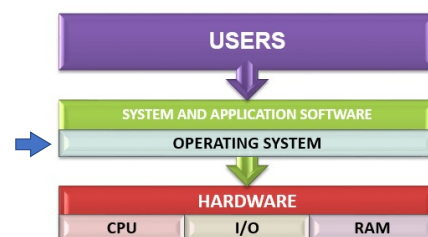
## Com SO

Detalhes genéricos de cada HW estão no SO

Maior compatibilidade

Menor esforço de programação

Aplicações centram-se no problema a resolver



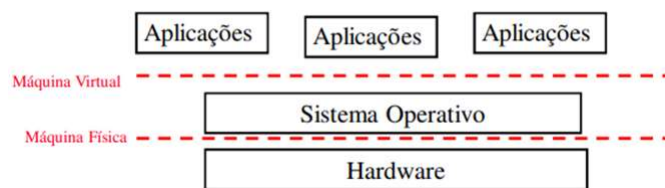
16

16



## Para que serve um SO?

- Criar uma espécie de máquina virtual sobre a máquina física, que disponibiliza objetos que permitem o desenvolvimento e execução de aplicações
- Independente do hardware onde se executa
- Tornar o sistema mais simples e flexível
- Disponibilizar o uso controlado, eficiente, partilhado, protegido do SW e HW



17

17

## Então o que é um SO?

O SO é um conjunto de módulos com as funções de:

- **Interface** entre utilizador e HW (tornar simples)
- **Gestor** de Recursos (maximizar rendimento)
  - CPU
  - Memória principal
  - Memória secundária
  - Dispositivos
    - Discos
    - Impressoras
    - ...

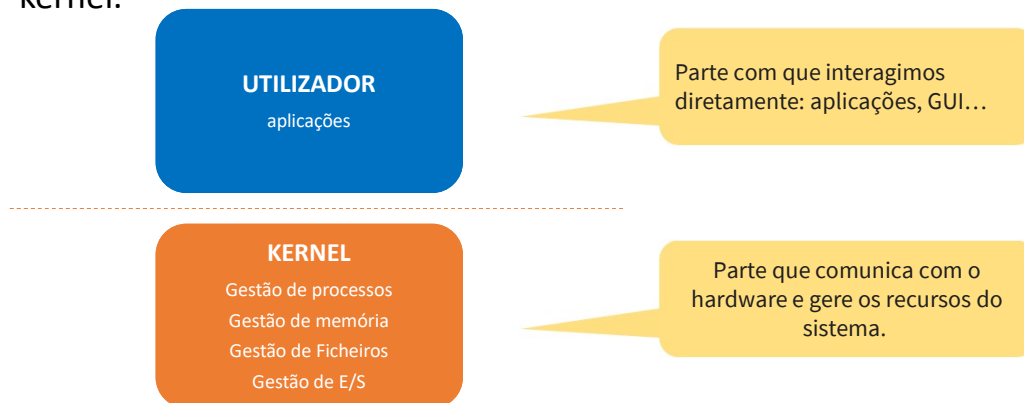
∴ permite-nos  
interagir com o  
sistema e faz a  
gestão dos recursos  
de forma eficiente

18

18

# Então o que é um SO?

Um SO tem dois componentes principais: o modo utilizador e o modo kernel.



19

19

## Proteções disponibilizadas pelo SO

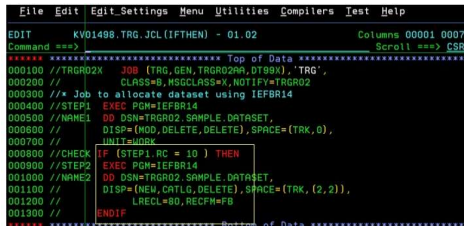
- Dois modos de operação
  - **Modo Utilizador:** permite a execução de instruções comuns (aritmética e lógica)
  - **Modo Núcleo:** zona protegida onde tudo pode ser executado.
- Proteção E/S
  - Instruções de E/S são privilegiadas
- Proteção da Memória
  - Cada programa só consegue aceder à memória que lhe foi destinada
- Proteção do CPU
  - O CPU deve ser gerido pelo SO, mesmo que haja muitas solicitações das aplicações

20

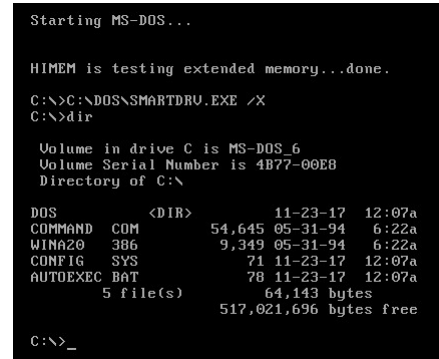
20

## Interação com o SO – pelo utilizador

- Interação na forma de texto



```
File Edit Edit Settings Menu Utilities Compilers Test Help
EDIT KV01488.TRG.JCL (IFTHEN) - 01.02 Columns 00001 00072
Command ==>
***** ***** Top of Data *****
000100 //TRGRD2X JOB (TRG,GEN,TRGRD2X,DT89X),TRG',
000200 // CLASS=8,MSGCLASS=X,NOTIFY=TRGRD2
000300 // * Job to allocate dataset using IEFBR14
000400 //STEP1 EXEC PGM=IEFBR14
000500 //NAME1 DD DSN=TRGRD2.SAMPLE.DATASET,
000600 // DISP=(MOD,DELETE,DELETE),SPACE=(TRK,0),
000700 // UNIT=SDR
000800 //CHECK IF (STEP1.RC = 10 ) THEN
000900 //STEP2 EXEC PGM=IEFBR14
001000 //NAME2 DD DSN=TRGRD2.SAMPLE.DATASET,
001100 // DISP=(NEW,CATLG,DELETE),SPACE=(TRK,(2,2)),
001200 // LRECL=80,RECFM=FB
001300 //
001400 // ENDIF
***** ***** Bottom of Data *****
```



```
Starting MS-DOS...

HIMEM is testing extended memory...done.

C:\>C:\DOS\SMARTDRV.EXE /X
C:\>dir

Volume in drive C is MS-DOS_6
Volume Serial Number is 4B77-00E8
Directory of C:\

DOS             <DIR>             11-23-17   12:07a
COMMAND  COM             54,645  05-31-94   6:22a
WINA20    386              9,349  05-31-94   6:22a
CONFIG   SYS              71  11-23-17   12:07a
AUTOEXEC BAT              78  11-23-17   12:07a
          5 file(s)             64,143 bytes
                               517,021,696 bytes free

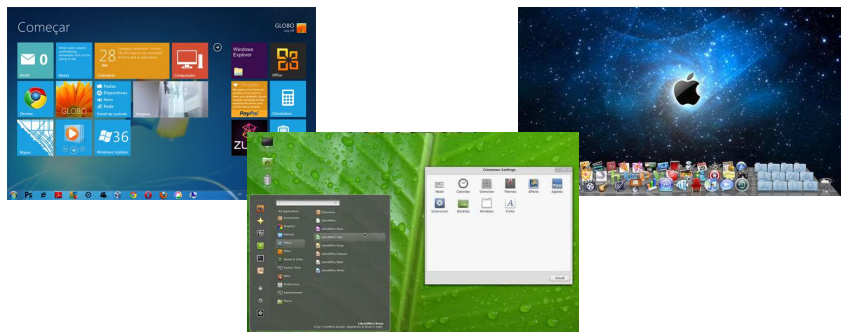
C:\>_
```

21

21

## Interação com o SO – pelo utilizador

- Interação gráfica

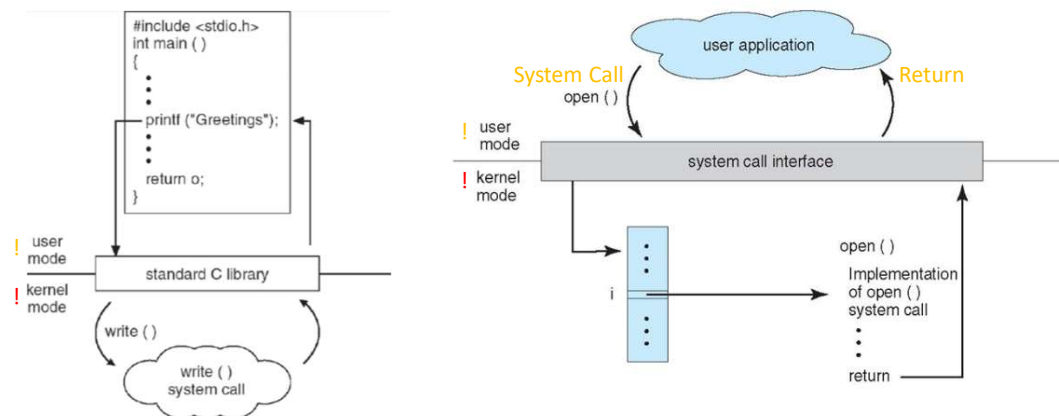


22

22

## Interação com o SO – por programas

- Interação com chamadas de sistema



23

23

## Interação com o SO – por programas

- System Calls – chamadas de sistema

|                         | Windows                                                                             | Unix                                   |
|-------------------------|-------------------------------------------------------------------------------------|----------------------------------------|
| Process Control         | CreateProcess()<br>ExitProcess()<br>WaitForSingleObject()                           | fork()<br>exit()<br>wait()             |
| File Manipulation       | CreateFile()<br>ReadFile()<br>WriteFile()<br>CloseHandle()                          | open()<br>read()<br>write()<br>close() |
| Device Manipulation     | SetConsoleMode()<br>ReadConsole()<br>WriteConsole()                                 | ioctl()<br>read()<br>write()           |
| Information Maintenance | GetCurrentProcessID()<br>SetTimer()<br>Sleep()                                      | getpid()<br>alarm()<br>sleep()         |
| Communication           | CreatePipe()<br>CreateFileMapping()<br>MapViewOfFile()                              | pipe()<br>shmget()<br>mmap()           |
| Protection              | SetFileSecurity()<br>InitializeSecurityDescriptor()<br>SetSecurityDescriptorGroup() | chmod()<br>umask()<br>chown()          |

24

24

# Análise de conceitos

Monoprogramado

Multiprogramado

Tempo Partilhado

Memória Virtual

...

25

25

## Monoprogramado vs Multiprogramado

- Sistema **Monoprogramado**

Um programa a correr que permanece na memória ate terminar (e.g.: DOS)

- Sistema **Multiprogramado**

Vários programas correm simultaneamente e partilham recursos (CPU, memória, ES...) e.g. Unix, Windows...

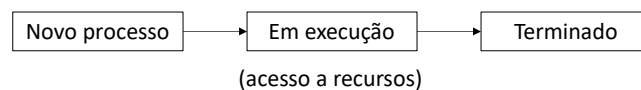
Vantagens?

26

26

# Monoprogramado

- O CPU, memória e periféricos permanecem exclusivamente dedicados à execução de um único programa
- OS recursos são pouco rentabilizados (desvantagem...porquê?)
- Fácil implementação e execução (vantagem...porquê?)

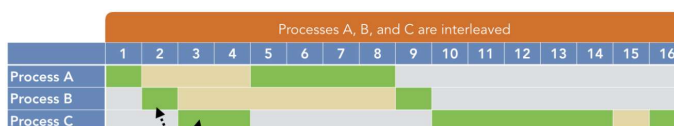


27

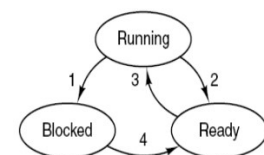
27

# Multiprogramado

- Vários programas dividem os recursos do sistema
- Enquanto uma tarefa bloqueia, outra é executada\*
- Maior eficiência e produtividade
- Redução de custos
- Exige mecanismos de gestão processos



\* CPU executing processes B and C while A waits on I/O



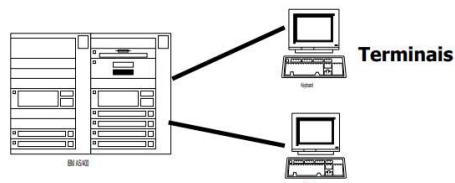
1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

28

28

## Tempo Partilhado

- Na década de 60 surge a abordagem de partilha de tempo do computador pelos vários terminais  
e.g. sistemas de reservas, sistema bancário
- Cria a ilusão que o computador está permanentemente disponível para o utilizador.

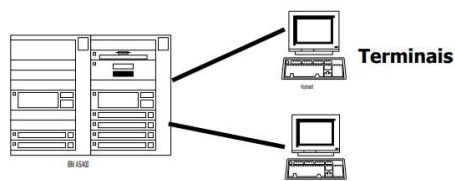


29

29

## Consequências do Tempo Partilhado

- Revisão dos algoritmos de escalonamento
- Definição de mecanismos de segurança
- Aparecimento dos sistemas de ficheiros

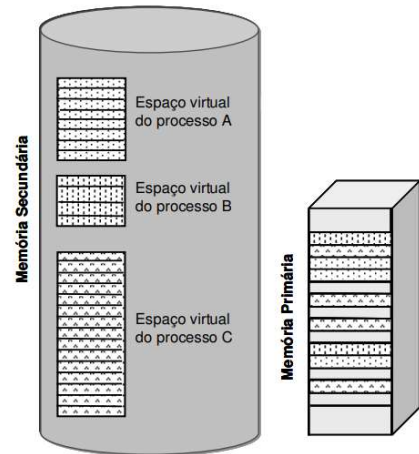


30

30

## Memória Virtual – Gestão de Recursos

- Elimina a restrição física imposta pelo tamanho da memória física (swapping)
- Permitindo um grau de multiprogramação muito superior



31

31

## Tipos de SO

32

32



## Sistemas Proprietários vs. Abertos

- 1970s: Havia vasta oferta de SOs de tempo partilhado (Unix, VMS, OS390,...)
- **Todos eles sistemas proprietários:**
  - Tinham restrições (legais e técnicas) na sua utilização e cópia
  - Desenvolvidos pelo fabricante de hardware
  - Código assembly, não divulgado
  - **Gera dependências:** aplicação / SO / Plataforma

33

33

## Sistemas Abertos

- Sistema portátil e interoperável
- Oferece uma interface normalizada
- Filosofia alavancada pelo Unix
  - Divulgação do código fonte
  - Interfaces de programação standard
  - Encorajava o desenvolvimento de SW e HW por terceiros
  - Linux – software livre, open-source, segue a licença GPL

34

34

## Tempo Virtual vs. Tempo Real

- Tempo virtual:
  - O tempo de execução dos programas **não tem relação com o tempo cronológico exterior** ao computador.
  - São os sistemas habituais **utilizados na maioria dos computadores** quer os que se executam nas máquinas clientes, quer nos servidores (ex: Windows, Linux, MAC/OS)
- Tempo real:
  - Garantir que o computador **produz uma resposta a um acontecimento externo** num intervalo de tempo determinado, caso contrário o sistema não cumpre a sua especificação – falha (sensores / atuadores)
  - e.g. piloto automático, produção industrial...

35

35

## Sistemas Embebidos – O que são?

- Software integrado com o hardware
- O sistema não oferece interface para desenvolver aplicações
- E.g.
  - Symbian, utilizado nos telefones Nokia
  - Windows CE, usado nos Smartphones e pocket PCs
  - PalmOS , usado nos PDA



36

36

# Estruturas de SO

Monolítica  
Sistema em camadas  
Micronúcleo  
Máquina virtual

37

37

## Estrutura Monolítica

- Um único bloco com todos os módulos
  - executado em modo núcleo
- Forma mais primitiva de SO
- Proteção entre modo utilizador e modo núcleo
- +rápido



38

38

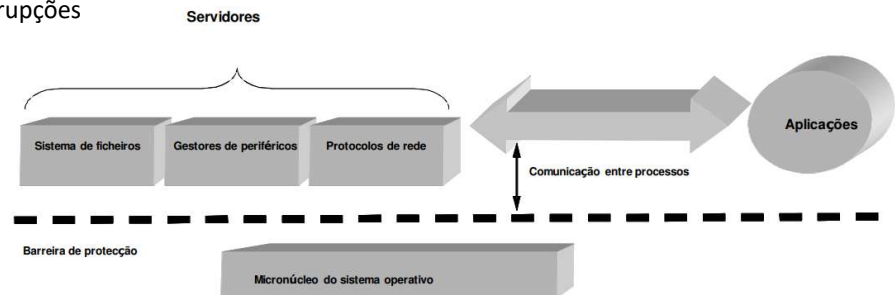
## Estrutura Micronúcleo

- Micronúcleo só com o essencial do SO:

- Gestão de fluxos de execução – threads
- Gestão dos espaços de endereçamento
- Comunicação entre processos
- Gestão das interrupções

- +fiável +seguro

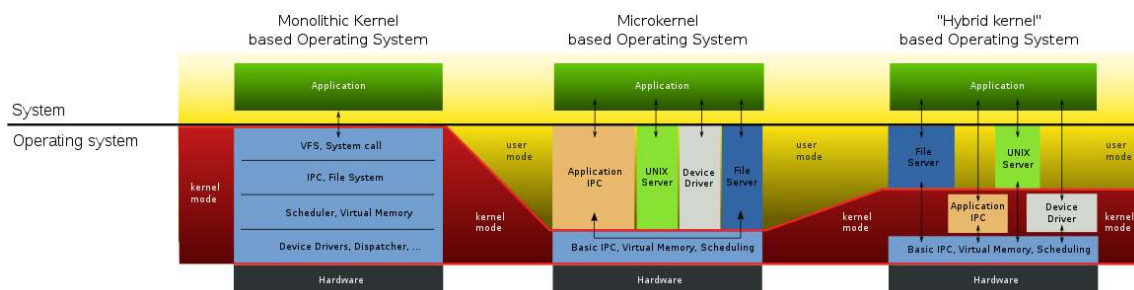
- Desvantagem?



39

39

## Monolítico vs. Micronúcleo



40

40

## Sistemas em Camadas

- Camadas hierárquicas (cada camada usa serviços da inferior)
- Fácil modificar código de uma só camada
- Mecanismos de proteção
- Maior segurança
- Desvantagem?

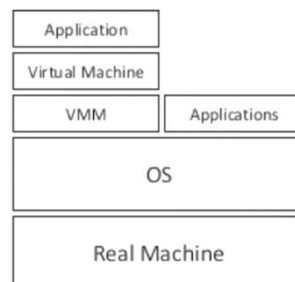


41

41

## Máquina Virtual

- Software que **executa aplicações e núcleos de SO** de forma mediada (parecendo que os executa numa máquina real)
- Usa os recursos da máquina real



42

42